PolyMorph: Learning Locomotion Policy for Cross-Embodiment Adaptation

Hyunsik Oh, Seunghun Jeon, Donghoon Youm, Jungwoo Hur, Hyeongjun Kim,
Jeong Hyun Lee, Moonkyu Jung, and Jemin Hwangbo*

arXiv:2610.19615v1 [cs.RO] 14 Oct 2026

***Abstract*—Reinforcement learning has enabled robust locomotion for legged robots, but learned policies are typically tied to a specific robot morphology and require additional engineering when the number of legs, actuators, or mechanical properties change. We present PolyMorph, a morphology-conditioned framework that learns a single locomotion policy for heterogeneous legged robots. PolyMorph converts each robot's URDF into joint-link modules and combines a recurrent encoder, graph-based Transformer, and per-joint decoder to generate actions across variable leg counts and joint configurations. To improve training stability, PolyMorph uses morphology-normalized rewards, adaptive robot sampling, and URDF-derived PD-gain initialization. We evaluate PolyMorph in simulation across bipedal, tripedal, and quadrupedal robots, including unseen morphology variants and platforms, and deploy the policy on real quadruped platforms operated in multiple configurations. The results show that a single PolyMorph policy generalizes across diverse robot morphologies without per-robot policy retraining.**

***Index Terms*—Reinforcement Learning, Multi-Embodiment Learning, Legged Robots**

I. Introduction

In the field of robotics, significant progress has been made through various control methodologies, including Model Predictive Control (MPC) [1] and learning-based approaches such as imitation learning (IL) [2], [3] and reinforcement learning (RL) [4], [5]. Among these, model-free RL has become a powerful tool for legged robot control because it directly optimizes policies from environment interactions without requiring explicit dynamics models. This makes RL particularly effective for complex and underactuated systems, enabling robust locomotion over challenging terrains such as rough slopes, stairs, and deformable terrain [6], [7].

Most existing studies on legged locomotion focus on enabling a single platform to perform a specific task [8], [9]. However, policies trained for one robot often fail to transfer to another due to differences in morphology, including degrees of freedom, link masses, inertias, and kinematic configurations. Adapting a policy to a new robot typically requires redesigning the environment, observations, action space, and reward functions, which demands substantial engineering effort. This motivates the development of general controllers that can operate across diverse legged robots.

In this work, we address the following question: *Can we train a single locomotion policy that operates across legged robots with diverse morphologies, including different numbers of legs and actuators, and deploy it on real hardware without per-robot retraining?* We argue that such a controller must not only observe the robot's current state but also reason about its morphology, and that achieving this in practice requires addressing training-level imbalances that arise when robots with very different reward magnitudes and learning difficulties are trained together.

To this end, we propose PolyMorph, a cross-embodiment locomotion framework with a URDF-conditioned policy interface. As shown in Fig. 1, the same policy is deployed across diverse legged morphologies on various rough terrains in both simulation and the real world. Each robot is represented as a sequence of joint-link modules parsed from its URDF, and padding with valid-action masking allows the same policy to handle variable leg counts, actuator numbers, and joint configurations without redesigning the action interface. The policy combines a recurrent state encoder that summarizes proprioceptive history, a Transformer that conditions on URDF-derived graph structure, and per-joint decoder heads that generate actions only for actuated joints.

A unified locomotion policy across heterogeneous morphologies can shorten the robot design-test cycle by allowing new designs or modified configurations to be evaluated without retraining a separate controller for each platform, which is particularly useful for rapid robot design workflows [10], [11]. It also enables policy reuse across heterogeneous robot platforms and exposes the policy to broader morphology and dynamics variations.

Beyond designing a policy that can process robots with different kinematic structures, we observe that jointly optimizing across morphologies introduces training-level obstacles: reward magnitudes can scale with the number of joints and legs, biasing optimization toward certain morphologies; easy-to-learn robots can dominate uniform sampling and reduce the learning signal for harder morphologies such as bipeds; and manually tuned PD gains reintroduce per-platform engineering effort. Complementary to prior multi-embodiment methods that emphasize morphology-conditioned policy architectures, PolyMorph addresses these training-level obstacles through morphology-normalized rewards, adaptive robot sampling, URDF-derived PD-gain initialization, together with curriculum learning and domain randomization for robust training. The same policy parameters are shared across all robots, and only morphology-dependent inputs are derived from each robot's URDF.

In summary, the main contributions of this work are as follows:

- We identify and address training-level obstacles in cross-embodiment optimization, including reward-scale mismatch, sampling imbalance between easy and hard

Fig. 1. **A single policy for cross-embodiment legged locomotion.** PolyMorph is trained on diverse legged robots in simulation (left), zero-shot transfers to unseen morphology variants and platforms (top-right), and is deployed on real-world robots in multiple configurations (bottom-right).

robots, and per-platform gain tuning, through morphology-normalized rewards, adaptive robot sampling, and URDF-derived PD-gain initialization.
- We adapt joint-link tokenization, embodiment-masked attention, recurrent proprioceptive encoding, and per-joint action decoding into a URDF-conditioned policy interface that handles variable leg counts, actuator numbers, and kinematic structures with shared parameters.
- We validate PolyMorph on heterogeneous legged robots in simulation and demonstrate cross-morphology hardware deployment on two real quadruped platforms operated in quadrupedal, tripedal, and bipedal configurations, with additional ablations and comparisons against multi-embodiment baselines.

II. Related Work

A. Transformer architectures for robot control

In legged robot control, Transformer-based policies have been used to process observation histories or sensory inputs for locomotion [12]. Beyond such temporal use, Transformers also model relationships among input tokens through self-attention [13], making them suitable for control problems where multiple body parts must interact. This spatial formulation is particularly attractive for multi-embodiment locomotion, since variable-length token sequences naturally correspond to robots with different numbers of joints. However, standard Transformers do not explicitly encode the physical structure of the robot and must infer morphology-dependent relationships only from data, which can be sample-inefficient when training across heterogeneous robots. Body Transformer [14] uses the robot's adjacency graph as an attention mask to inject structural bias for a fixed morphology. PolyMorph adopts this structural prior in a policy shared across heterogeneous morphologies, where URDF-derived morphology inputs and attention masks vary across embodiments and are combined with recurrent state encoding.

B. Cross-embodied policy learning

Cross-embodied policy learning aims to train a single policy that can operate across different robot embodiments. In legged robotics, Metamorph [15] uses a Transformer to control modular robots but is evaluated only in simulation. GenLoco [16] achieves real-world transfer through morphology randomization but focuses on quadrupeds. URMA [17] learns multi-embodiment locomotion across quadrupedal and bipedal robots with an attention-based encoder–decoder. Graph neural networks [18] encode robot morphology as graphs. GCNT [19] further combines graph-based morphology encoding with Transformer communication to learn controllers across different robot structures. LocoFormer [20] takes a different approach, combining large-scale procedural training with long-context adaptation to generalize locomotion across unseen embodiments. While these works have advanced policy architectures, the training-level aspects of jointly optimizing heterogeneous legged robots have received less attention. PolyMorph also targets them directly. Beyond locomotion, Chen et al. [21] and Gupta et al. [22] study cross-robot transfer from demonstrations through intention-aligned representations and kinematics-aware policy synthesis, respectively.

III. Method

We propose PolyMorph, a morphology-aware locomotion learning framework that controls legged robots with different morphologies using a single policy. For each robot k, we formulate the control problem as a Markov decision process (MDP) $\mathcal{M}^k = (\mathcal{S}^k, \mathcal{A}^k, T^k, R^k, \gamma)$. Given a set of K robots with different morphologies, our objective is to learn one policy $\pi_\theta(a_t^k | s_t^k)$ that maximizes the average expected return:

$$\max_\theta \frac{1}{K} \sum_{k=1}^{K} \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{H} \gamma^t r_t^k \right]. \tag{1}$$

We optimize the policy using Proximal Policy Optimization (PPO) [5].

As shown in Fig. 2, PolyMorph consists of three components: a state context encoder, a morphology-aware Transformer,

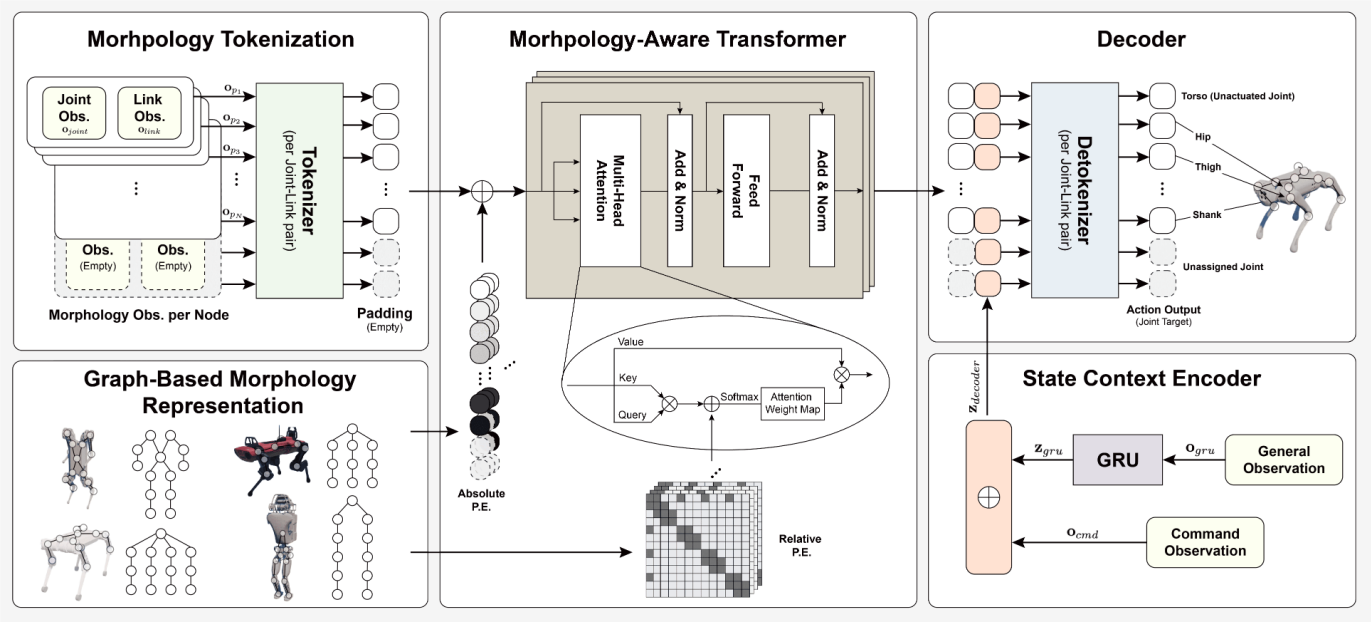

Fig. 2. **Overall architecture of PolyMorph.** PolyMorph consists of a GRU-based state context encoder, a morphology-aware Transformer that processes joint-link tokens using graph-based attention, and a decoder that generates actions for each actuated joint.

and a decoder. The state context encoder processes general proprioceptive observations over time. The morphology-aware Transformer represents each robot as a sequence of joint-link modules and extracts morphology-conditioned embeddings using graph-based attention. The decoder combines the state embedding, morphology embedding, and command input to generate target actions for each actuated joint.

A. State Context Encoder

The state context encoder uses a shared GRU to summarize proprioceptive history that is important for locomotion across robot morphologies. While each robot has different kinematic structure and actuator count, quantities such as body orientation, angular velocity, joint states, and previous target actions provide temporal information that is useful for balance, contact transitions, and recovery from disturbances. We therefore process these observations with a recurrent encoder and provide the resulting latent vector to the action decoder together with the morphology-conditioned Transformer features.

The actor observation is defined as

$$\mathbf{o}_{t,\mathrm{GRU}} = \left[\mathbf{g}_t,\ \boldsymbol{\omega}_t,\ \mathbf{q}_t,\ \dot{\mathbf{q}}_t,\ \mathbf{q}_{t-1}^{des}\right], \tag{2}$$

where $\mathbf{g}_t$ is the roll-pitch vector of the body, $\boldsymbol{\omega}_t$ is the body angular velocity, $\mathbf{q}_t$ and $\dot{\mathbf{q}}_t$ are the joint positions and velocities, and $\mathbf{q}_{t-1}^{des}$ is the previous target joint position. These observations are obtained from the IMU and joint encoders. For robots with different numbers of joints, we use a configurable maximum number of actuated joints, $N_{\max} = 15$ in the current experiments, and zero-pad the joint-related observations for robots with fewer joints. $N_{\max}$ can be increased for higher-DoF robot sets.

The GRU outputs a latent vector $\mathbf{z}_{\mathrm{gru}}$, which provides temporal context for action generation. We use an asymmetric actor-critic structure: the critic receives additional privileged information available in simulation, including body height, body linear velocity, and terrain height scan points, while the actor uses only onboard proprioceptive sensing at deployment.

B. Morphology-Aware Transformer Network

To handle robots with different numbers of joints and different kinematic structures, PolyMorph represents each robot as a sequence of joint-link modules. Each module consists of one joint and its attached child link, and is used as one Transformer token. The token observation is

$$\mathbf{o}_{p_i} = \left[\mathbf{o}_{link_i},\ \mathbf{o}_{joint_i}\right], \tag{3}$$

where $\mathbf{o}_{link_i}$ includes link mass, inertia, and center-of-mass position, while $\mathbf{o}_{joint_i}$ includes joint position, nominal joint angle, recent joint states, recent target actions, and PD gains. These quantities are either parsed from the URDF or updated from simulation and sensor observations.

The token sequence is ordered by traversing the robot's kinematic tree from the root body. Since robots may have different numbers of modules, unused token slots are filled with zero padding. Each token is projected through an independent linear layer and combined with an absolute positional embedding:

$$\mathbf{x}_i = \phi_i(\mathbf{o}_{p_i}; W_{e_i}) + \mathbf{w}_{pos,i}, \tag{4}$$

where W_{e_i} is the token-specific projection parameter and $\mathbf{w}_{pos,i}$ encodes the position of the module in the ordered kinematic sequence. This absolute positional embedding provides the Transformer with information about the module order.

To inject morphology into the attention mechanism, we follow the embodiment-masked attention idea of Body Transformer [14] and use the URDF-derived robot graph as a relative positional mask. Unlike a fixed-morphology setting, PolyMorph constructs joint-link tokens for each robot, handles different numbers of modules through padding, and later applies actions

only to valid actuated joints. The self-attention operation is defined as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + r\right)V, \quad (5)$$

where Q, K, and V are query, key, and value matrices, d_k is the token dimension, and r is the morphology-aware attention bias. We construct r from the adjacency matrix M [14] of the robot morphology graph:

$$r_{i,j} = \begin{cases} 0, & \text{if } M_{i,j} = 1, \\ -\infty, & \text{if } M_{i,j} = 0. \end{cases} \quad (6)$$

Here, $M_{i,j} = 1$ indicates that two modules are physically connected or allowed to attend to each other. This relative positional mask prevents attention to invalid structural connections and encourages the Transformer to reason over physically meaningful relationships. In this way, absolute positional embeddings encode the ordered kinematic sequence, while relative graph-based masking provides morphology-specific structural bias.

C. Decoder Network

The decoder generates actions by combining the morphology embedding, state embedding, and command input. For each module i, the decoder input is

$$\mathbf{z}_{i,\text{dec}} = [\mathbf{z}_{i,\text{trans}},\ \mathbf{z}_{gru},\ \mathbf{o}_{cmd}], \quad (7)$$

where $\mathbf{z}_{i,\text{trans}}$ is the Transformer output for module i, $\mathbf{z}_{gru}$ is the GRU output, and $\mathbf{o}_{cmd}$ is the desired velocity command.

For the actor, each module-specific decoder outputs the mean of a Gaussian action distribution:

$$\pi_\theta(a_i^k | s^k) = \mathcal{N}(\mu_i(\mathbf{z}_{i,decoder}; W_{d_i}),\ \Sigma), \quad (8)$$

where W_{d_i} is the independent decoder parameter for module i, and Σ is a diagonal covariance matrix. Action samples from this distribution are applied only to the actuated joint parts corresponding to the specific k-th robot, excluding the first module (index 0), which represents the root element and is not actuated.

For the critic, the same decoder structure outputs a value estimate for each valid module. The final value is computed by averaging over the valid root and actuated modules:

$$V(s^k) = \frac{1}{N+1}\sum_{i=0}^{N} V_i, \quad (9)$$

where V_i is the value for the i-th module, N is the number of actuated joints, and the summation starts from the root module at index 0. The resulting value function is used in PPO to compute the advantage and update the actor and critic.

D. Training Techniques for Generalization

When a single policy is trained to control robots with heterogeneous morphologies, three practical bottlenecks arise simultaneously: unequal learning difficulty across robots, morphology-dependent reward scales, and per-platform PD-gain tuning. PolyMorph integrates adaptive robot sampling, morphology-normalized rewards, and URDF-derived gain initialization to address these bottlenecks, respectively.

Adaptive robot sampling. Uniformly sampling robots during training can under-train difficult morphologies such as bipeds. To balance learning, we increase the sampling probability of robots with lower rewards. The probability of sampling robot i is computed as

$$P_i = \frac{\exp(\bar{r} - r_i)}{\sum_j \exp(\bar{r} - r_j)}, \quad (10)$$

where r_i is the reward of robot i and $\bar{r}$ is the average reward over all robots, and the denominator ensures the probabilities sum to 1. To avoid overfitting to difficult robots, half of the environments are still sampled uniformly.

Reward normalization. When learning a robot-agnostic policy, the reward should encourage motion generation and command tracking without favoring a specific robot morphology. However, if reward magnitudes depend strongly on the number of joints or legs, the policy can converge to a local optimum where only certain robot configurations achieve high rewards. Large reward-scale differences across robots can also destabilize PPO training and may even cause learning failure. To avoid this, we normalize morphology-dependent reward terms by the number of physical components that generate them. Joint-related penalties, such as joint torque, position, velocity, acceleration, and action smoothness, are divided by the number of actuated joints. Similarly, leg- or foot-related terms, such as airtime and foot clearance, are divided by the number of legs. This keeps reward scales consistent across morphologies and helps the policy learn robust behaviors without bias toward a particular robot configuration.

Automatic PD-gain design. The policy outputs desired joint positions, which are converted to torques through a standard PD controller. To avoid robot-specific gain tuning in multi-embodiment training, we use a simple morphology-aware gain rule based on the URDF-derived mass matrix evaluated at a nominal posture.

We define a robot-level inertia scale from the URDF-derived mass matrix $\mathbf{M}(\mathbf{q}_0)$ at the nominal posture $\mathbf{q}_0$ as

$$\lambda = \sum_{i=1}^{3} M_{ii}(\mathbf{q}_0), \quad (11)$$

which provides a morphology-dependent quantity for gain scaling.

The proportional gain is computed as

$$K_p = \alpha^2 \lambda, \quad (12)$$

where α is a robot-agnostic scaling coefficient. We set $\alpha = 0.8$ for all robots, which yields K_p values in a practical range for the considered robots. The resulting scalar is applied uniformly to all actuators, i.e., $\mathbf{K}_p = K_p\mathbf{I}$. This rule replaces per-platform manual tuning with a single shared coefficient and a robot-specific inertial scale derived from the URDF.

For the derivative gain, we first compute a joint-wise damping value using the standard second-order damping relation:

$$K_{d,i} = 2\zeta\sqrt{M_{ii}(\mathbf{q}_0)K_p}, \quad (13)$$

where $M_{ii}(\mathbf{q}_0)$ is the diagonal joint-space inertia term of joint i and ζ is the damping ratio. We set $\zeta = 0.8$ to obtain a slightly underdamped response with small overshoot, balancing response speed and stability. Since $K_{d,i}$ can vary significantly between heavy proximal joints and light distal joints, directly using joint-wise values may introduce excessive damping variation. Therefore, we use the median value as a shared derivative gain:

$$K_d^* = \text{median}\{K_{d,i}\}, \qquad \mathbf{K}_d = K_d^* \mathbf{I}. \tag{14}$$

By grounding the gain scale in URDF-derived inertial quantities, the proposed rule provides morphology-aware PD gains that are computed once from $\mathbf{M}(\mathbf{q}_0)$ and applied across different robots without manual per-platform tuning.

Adjusting the Level of Difficulty We gradually increase task difficulty during training. A curriculum factor c_k is updated as

$$c_{k+1} = c_k^{\gamma}, \qquad 0 < \gamma < 1, \tag{15}$$

and environment parameters are interpolated between easy and hard settings:

$$p(k) = p_{\text{easy}} + c_k(p_{\text{hard}} - p_{\text{easy}}). \tag{16}$$

This schedule is applied to command ranges, terrain roughness, external disturbances, reward weights, and morphology perturbations. We also use a cosine-annealing learning-rate scheduler with warmup restarts to encourage exploration at the beginning of each curriculum stage and stable convergence afterward.

Domain randomization. To improve sim-to-real transfer and robustness, we randomize both environmental and morphology-related parameters during training. These include observation noise, link and joint parameters, PD gains, control latency, friction, and external disturbances. The randomization is applied at the beginning of each episode or at every time step depending on the parameter.

IV. Results

Our experiments evaluate whether the overall PolyMorph framework enables stable training across heterogeneous robots, whether a single policy achieves locomotion across seen and unseen morphologies and transfers to real hardware, and whether PolyMorph improves performance relative to alternative architectures and multi-embodiment baselines.

A. Implementation Details

Training was conducted in RaiSim [23] using 350 parallel environments for 20,000 iterations on an AMD EPYC 9654 CPU and an NVIDIA RTX 4090 GPU, requiring approximately 2.7 days. We trained three random seeds and evaluated each method on the same 100 episodes per robot. The deployed actor contains approximately 0.33 million parameters. For real-world deployment, the 100 Hz policy was exported to ONNX format and executed on the onboard computers of Raibo1 and Raibo2, with an average latency of 0.366 ms, as shown in Fig. 3.

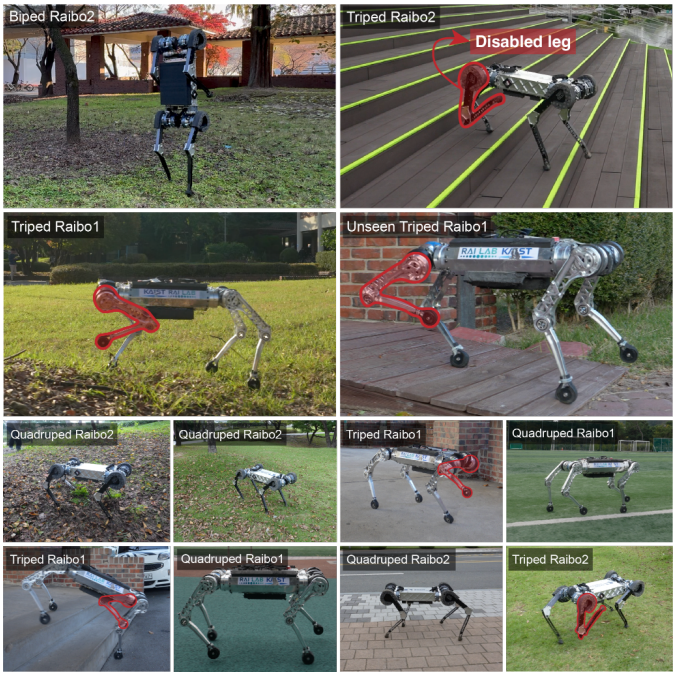

Fig. 3. A single trained controller is evaluated across diverse legged robot morphologies in simulation and deployed on Raibo1 and Raibo2 in bipedal, tripedal, and quadrupedal configurations. Both real-world deployment and unseen morphology configurations are tested on rough terrain.

TABLE I
COMPARISON BETWEEN AUTOMATIC AND MANUAL PD GAINS

Robot	Automatic method		Manual reference	
	K_p	K_d	K_p	K_d
Raibo1	51	1.3	50	0.5
Raibo2	77	2.7	80	1.0
Hound	81	2.2	80	1.0
Aliengo	40	1.2	40	2
Anymal-C	100	2.0	80	2.0
Atlas	260	3.6	300	4.0
Go1	25	0.6	30	0.7
Mini-Cheetah	21	0.8	20	0.5

TABLE II
REWARD-NORMALIZATION ABLATION.

Category	Metric	Full	w/o norm.
Early scale bias	Joint-count corr. $\lvert r\rvert \downarrow$	**0.256**	0.904
Scale imbalance	Joint-reward CV $\downarrow$	**0.136**	1.370
	Foot-reward CV $\downarrow$	**0.527**	1.413
Outcome	Eval. return $\uparrow$	**5.024**	1.109
	Avg. termination rate (%) $\downarrow$	**0.030**	1.556
	Termination-rate gap (pp) $\downarrow$	**0.059**	4.653

B. Framework Evaluation

We trained a single locomotion policy across diverse legged robots, including Raibo1, Raibo2, Hound, Aliengo, Anymal-C, Atlas, Cassie, Go1, Laikago, Mini-Cheetah, and multiple morphology variants. The training set covers bipedal, tripedal, and quadrupedal configurations with different kinematic and dynamic properties, with masses ranging from 11kg to 135kg. Tripedal variants are generated by deleting one leg; FR, FL, HR, and HL denote the removed front-right, front-left, hind-right, and hind-left legs, respectively. The notation "fix" indicates that the corresponding leg is kept fixed instead of being removed.

All morphologies are trained with the same policy, reward

formulation, and training pipeline. Table I shows that the URDF-derived PD gains are broadly comparable to manual reference gains, while requiring no robot-specific tuning. The same gain coefficients are also used without retuning for the unseen B2 and Solo12 evaluations in Table IV, providing additional validation beyond the training platforms. All results use these automatic gains.

Table II ablates morphology-normalized rewards by removing the actuator- and leg-count normalization factors while keeping all other settings unchanged. Since the training reward scales differ between the two variants, Eval. return is computed using the same evaluation reward function for both policies. Joint-count corr. measures the early-training correlation between joint-related reward scale and the number of actuated joints, while Joint-reward CV and Foot-reward CV measure final reward-scale variation across morphology groups. Without reward normalization, reward scales become more morphology-dependent, resulting in lower evaluation return and a larger termination-rate gap. These results support morphology-normalized rewards for balanced policy optimization across heterogeneous legged robots.

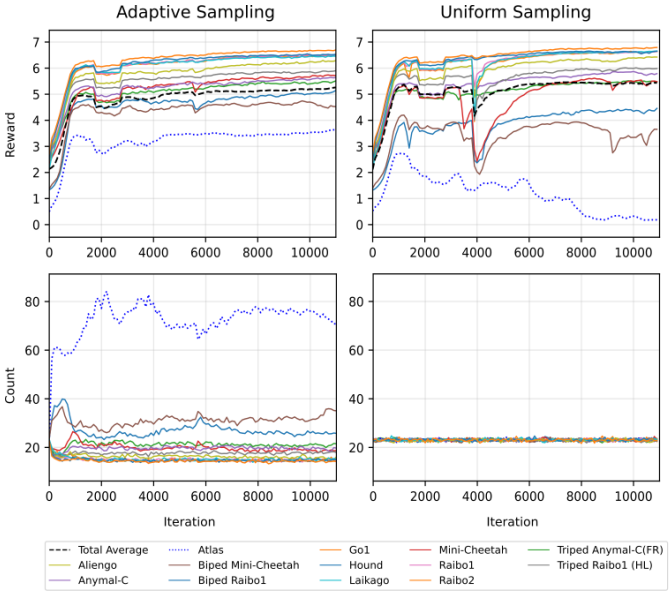

Fig. 4. Comparison of adaptive and uniform robot sampling strategies. The figures show the reward and the number of generations for each robot.

TABLE III
REAL-WORLD BLIND-LOCOMOTION EVALUATION.

Configuration	Robot	Trials	Success
Quadruped	Raibo1, Raibo2	20	20
Triped	Raibo1, Raibo2	20	20
Biped	Raibo2	10	9
Unseen triped	Raibo1	10	7

Since easy-to-learn morphologies such as quadrupeds can dominate multi-robot training, harder morphologies such as bipeds may receive insufficient updates under uniform sampling. As shown in Fig. 4, Atlas under uniform sampling fails to learn, with its reward decreasing as training progresses, while adaptive sampling allocates proportionally more environments to Atlas

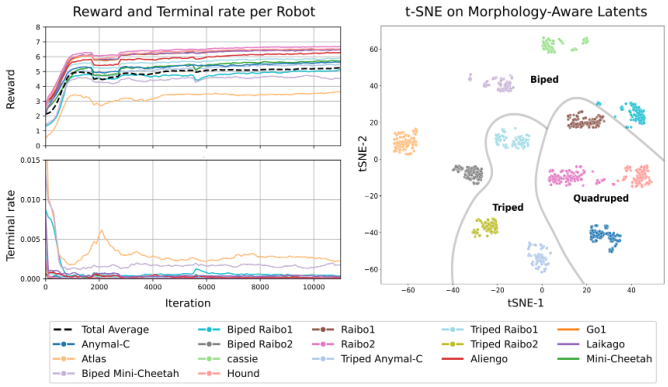

Fig. 5. The left panel shows rewards and termination rates. The right panel shows the t-SNE visualization of morphology-aware embeddings across bipedal, tripedal, and quadrupedal morphologies.

and restores stable convergence. This balances learning across diverse morphologies under limited computational resources.

We deployed the trained controller on Raibo1 and Raibo2 in quadrupedal, tripedal, and bipedal configurations, as shown in Fig. 3. In the tripedal experiments, the disabled leg remained attached with its joints locked, retaining mass and inertia absent from the leg-removed training morphologies. The same predefined 50 m outdoor route, comprising flat ground, slopes, steps, and naturally uneven terrain, was used for all configurations; success required completion without falling, emergency stop, or operator intervention. Table III summarizes the results. Among the unsuccessful trials, failure modes differed between the unseen tripedal and bipedal configurations. For the unseen tripedal configuration, failures occurred when insufficient foot clearance caused the swing foot to catch on branches or raised terrain edges, while slip events were generally recovered. For the bipedal configuration, the failure resulted from severe balance loss, with point-foot contacts and a high center of mass limiting recovery. These hardware results demonstrate that a single PolyMorph policy can be deployed across multiple real robot configurations without per-configuration retraining.

TABLE IV
COMMAND-TRACKING RMSE ACROSS DIFFERENT ROBOT MORPHOLOGIES

	Robot Model	V_{xy} **[m/s]**	ω_z **[rad/s]**
Seen	Raibo1	0.327(±0.112)	0.176(±0.169)
	Triped Raibo1(HL)	0.142(±0.083)	0.085(±0.083)
	Hound	0.168(±0.059)	0.058(±0.057)
	Anymal-C	0.172(±0.117)	0.092(±0.072)
	Atlas	0.246(±0.105)	0.193(±0.185)
	Biped Mini-cheetah	0.225(±0.078)	0.154(±0.153)
	Cassie	0.204(±0.127)	0.153(±0.152)
	Go1	0.152(±0.042)	0.083(±0.080)
Unseen	Triped Hound(FL)	0.145(±0.073)	0.072(±0.060)
	Biped Go1	0.323(±0.174)	0.180(±0.163)
	Triped Anymal-C(HL)	0.270(±0.094)	0.068(±0.066)
	B2	0.124(±0.068)	0.168(±0.111)
	Solo12	0.254(±0.112)	0.124(±0.121)

C. Framework Analysis

We analyze PolyMorph from two perspectives: cross-morphology control performance and morphology-aware representation learning. Using training curves and command-tracking results, we first evaluate whether a single policy maintains stable performance across robots with different leg counts, body configurations, and mechanical properties. We then use t-SNE embeddings to examine whether the graph-based Transformer captures meaningful morphology-dependent structure.

As shown in Fig. 5, PolyMorph converges stably across diverse robot morphologies. Quadrupeds generally achieve higher rewards due to their greater static and dynamic stability, whereas bipeds obtain lower rewards because balancing with fewer contacts is more difficult on rough terrain. Nevertheless, the terminal rates decrease and remain low after training, indicating stable locomotion even for challenging morphologies.

For command-tracking evaluation, each robot was tested over multiple rough-terrain episodes with randomized velocity commands. The commands $v_x \sim \mathcal{U}(-1, 2)$ m/s, $v_y \sim \mathcal{U}(-1, 1)$ m/s, and $\omega_z \sim \mathcal{U}(-1, 1)$ rad/s were sampled independently and held for 10 s. The first 2 s were excluded as a transient, and the RMSE was computed over the remaining 8 s and averaged across evaluation episodes. Table IV summarizes the command-tracking performance on rough terrain. PolyMorph achieves low RMSE for both seen and unseen robots, showing that the learned controller generalizes beyond the training morphologies. This generalization is enabled by representing each joint-link pair as an individual token and using graph-based attention to condition actions on the robot structure. Real-world experiments further confirm that the same policy can be deployed on different configurations while maintaining stable locomotion using only proprioceptive observations.

Among the unseen entries in Table IV, B2 and Solo12 evaluate quadrupedal extrapolation beyond the training set, which spans 11kg (Mini-Cheetah) to 52kg (Anymal-C). B2 (70kg) and Solo12 (2.5kg) represent heavy- and light-side extrapolation cases, respectively. PolyMorph achieves command-tracking accuracy on B2 comparable to seen platforms, while on Solo12, whose mass is less than one quarter of the lightest training quadruped and whose body scale and actuator characteristics differ substantially, it maintains command-following behavior with reduced tracking precision. Overall, PolyMorph transfers reliably across the quadrupedal training distribution and generalizes to morphologically similar out-of-distribution platforms, while extreme scale differences remain a direction for broader training coverage.

To examine whether the graph-based Transformer captures morphology information, we visualize the learned token embeddings using t-SNE. As shown in Fig. 5, the embeddings form clusters according to the number of legs and robot body structure. Robots with similar size and kinematic properties, such as Raibo2 and Hound, appear close to each other, while different configurations of the same robot form nearby but distinct clusters. Unseen robots also cluster near morphologically similar robots, suggesting that PolyMorph encodes structural features rather than simply memorizing robot identities, and learns morphology-aware representations that support robust policy learning across diverse legged robots.

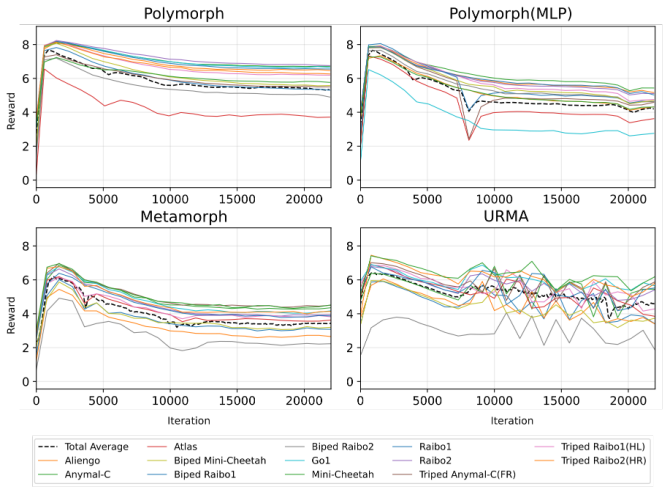

Fig. 6. Comparison of different networks. The figures show the total reward and per-robot reward for each network.

D. Ablation and Baseline Comparison

We evaluate PolyMorph from two perspectives. First, we examine structured token-wise processing by comparing PolyMorph with an MLP variant that flattens the joint-link features into a single vector instead of processing them as separate Transformer tokens, while keeping the same training pipeline. Second, we compare PolyMorph with representative multi-embodiment baselines, Metamorph [15] and URMA [17], under comparable training budgets. For controlled comparison, we preserve the method-specific observations, morphology representations, and network architectures of Metamorph and URMA following their original publications and official implementations, while training and evaluating all methods under identical training conditions, budget, and evaluation protocol. Each method is trained with 3 random seeds and evaluated under the protocol described in Sec. IV-A. In Table V, "Fail" indicates termination during evaluation (e.g., falling).

As shown in Fig. 6, PolyMorph achieves consistently higher total reward than the baselines and maintains balanced per-robot performance across diverse morphologies. The modular joint-link representation and graph-based Transformer attention allow the policy to decompose robot morphology into structured components and model their interactions, leading to more stable policy learning across heterogeneous robots.

The PolyMorph-MLP variant degrades in performance as the number and diversity of robots increase. This indicates that the graph-based Transformer is important for interpreting morphology and generalizing to new structures. Metamorph and URMA both rely on attention mechanisms but lack explicit morphology-aware structural bias and recurrent temporal processing of proprioceptive observations. As a result, Metamorph struggles to track commands and fails on some seen and unseen robots, while URMA remains competitive on some training robots but becomes unstable during later curriculum stages and fails on some unseen robots. These results suggest that attention-based architectures without explicit morphology-aware bias

TABLE V
COMMAND-TRACKING RMSE COMPARISON ACROSS DIFFERENT NETWORKS

	Robot	Polymorph		Polymorph(MLP)		Metamorph		URMA	
		V_{xy} [m/s]	ω_z [rad/s]	V_{xy} [m/s]	ω_z [rad/s]	V_{xy} [m/s]	ω_z [rad/s]	V_{xy} [m/s]	ω_z [rad/s]
Seen	Triped Raibo1(HL)	0.188(±0.100)	**0.098(±0.088)**	0.173(±0.083)	0.201(±0.106)	1.136(±0.111)	0.217(±0.160)	**0.172(±0.067)**	0.147(±0.147)
	Atlas	**0.479(±0.226)**	**0.204(±0.204)**	0.516(±0.195)	0.385(±0.236)	1.260(±0.145)	0.311(±0.194)	0.744(±0.234)	0.361(±0.286)
	Anymal-C	0.160(±0.091)	**0.082(±0.078)**	0.229(±0.101)	0.152(±0.110)	0.995(±0.139)	0.328(±0.201)	**0.107(±0.048)**	0.104(±0.067)
	Biped Mini-cheetah	**0.275(±0.103)**	**0.156(±0.144)**	0.409(±0.106)	0.240(±0.227)	0.612(±0.113)	0.617(±0.615)	**0.275(±0.077)**	0.314(±0.280)
Unseen	Triped Raibo2(HR-fixed)	**0.153(±0.103)**	**0.143(±0.143)**	- (fail)	- (fail)	0.611(±0.331)	0.630(±0.299)	0.256(±0.083)	0.176(±0.171)
	Triped Raibo1(HR)	0.484(±0.100)	0.195(±0.180)	**0.428(±0.076)**	**0.135(±0.126)**	- (fail)	- (fail)	- (fail)	- (fail)
	Hound	**0.244(±0.075)**	**0.108(±0.102)**	0.499(±0.094)	0.191(±0.092)	1.298(±0.058)	0.292(±0.159)	- (fail)	- (fail)
	Laikago	**0.090(±0.051)**	**0.160(±0.087)**	0.377(±0.059)	0.255(±0.147)	0.840(±0.231)	0.231(±0.174)	0.131(±0.090)	0.265(±0.209)

may not be sufficient to fit heterogeneous morphologies jointly. Together with the PolyMorph-MLP ablation, the comparison suggests that both morphology-aware graph attention and the training techniques contribute to PolyMorph's training stability, command tracking, and generalization to unseen morphologies.

V. CONCLUSION AND FUTURE WORK

In this work, we presented PolyMorph, a morphology-conditioned locomotion framework for controlling legged robots with diverse morphologies using a single policy. PolyMorph represents each robot as a URDF-derived sequence of joint-link modules and combines a recurrent state encoder, a graph-based Transformer, and per-joint decoder heads to generate actions across heterogeneous morphologies. Combined with morphology-normalized rewards, adaptive robot sampling, URDF-derived PD-gain initialization, curriculum learning, and domain randomization, the framework improves training stability across robots with different leg counts, configurations, and mechanical properties. Simulation and real-world experiments demonstrate robust command tracking on rough terrain across bipedal, tripedal, and quadrupedal robots, including unseen configurations of training platforms and unseen quadrupedal platforms.

The main limitation of the current framework is that generalization is reliable only within the morphology range covered by training and degrades under strong extrapolation in topology. Future work will focus on overcoming this limitation through robot data augmentation, such as procedurally perturbing URDFs and synthesizing morphology variants along the topology axis, so that the network is exposed to a broader range of robots during training and can extrapolate more reliably to platforms substantially different from the training set.

REFERENCES

[1] D. Youm, H. Jung, H. Kim, J. Hwangbo, H.-W. Park, and S. Ha, "Imitating and finetuning model predictive control for robust and symmetric quadrupedal locomotion," *IEEE Robotics and Automation Letters*, 2023.

[2] X. B. Peng, P. Abbeel, S. Levine, and M. Van de Panne, "Deepmimic: Example-guided deep reinforcement learning of physics-based character skills," *ACM Transactions On Graphics (TOG)*, vol. 37, no. 4, pp. 1–14, 2018.

[3] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," *arXiv preprint arXiv:2304.13705*, 2023.

[4] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[6] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Science robotics*, vol. 7, no. 62, p. eabk2822, 2022.

[7] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "Rma: Rapid motor adaptation for legged robots," *arXiv preprint arXiv:2107.04034*, 2021.

[8] H. Kim, H. Oh, J. Park, Y. Kim, D. Youm, M. Jung, M. Lee, and J. Hwangbo, "High-speed control and navigation for quadrupedal robots on complex and discrete terrain," *Science Robotics*, vol. 10, no. 102, p. eads6192, 2025.

[9] Y. Ma, A. Cramariuc, F. Farshidian, and M. Hutter, "Learning coordinated badminton skills for legged manipulators," *Science Robotics*, vol. 10, no. 102, p. eadu3922, 2025.

[10] J. H. Lee, H. Oh, J. Yoon, S.-J. Lee, T. Jin, J. Hwangbo, and S.-H. Bae, "Robotsketch: an interactive showcase of superfast design of legged robots," in *ACM SIGGRAPH 2024 Emerging Technologies*, 2024, pp. 1–2.

[11] Á. Belmonte-Baeza, J. Lee, G. Valsecchi, and M. Hutter, "Meta reinforcement learning for optimal design of legged robots," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 12 134–12 141, 2022.

[12] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, "Real-world humanoid locomotion with reinforcement learning," *Science Robotics*, vol. 9, no. 89, p. eadi9579, 2024.

[13] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.

[14] C. Sferrazza, D.-M. Huang, F. Liu, J. Lee, and P. Abbeel, "Body transformer: Leveraging robot embodiment for policy learning," *arXiv preprint arXiv:2408.06316*, 2024.

[15] A. Gupta, L. Fan, S. Ganguli, and L. Fei-Fei, "Metamorph: Learning universal controllers with transformers," *arXiv preprint arXiv:2203.11931*, 2022.

[16] G. Feng, H. Zhang, Z. Li, X. B. Peng, B. Basireddy, L. Yue, Z. Song, L. Yang, Y. Liu, K. Sreenath *et al.*, "Genloco: Generalized locomotion controllers for quadrupedal robots," in *Conference on Robot Learning*. PMLR, 2023, pp. 1893–1903.

[17] N. Bohlinger, G. Czechmanowski, M. Krupka, P. Kicki, K. Walas, J. Peters, and D. Tateo, "One policy to run them all: an end-to-end learning approach to multi-embodiment locomotion," *arXiv preprint arXiv:2409.06366*, 2024.

[18] T. Wang, R. Liao, J. Ba, and S. Fidler, "Nervenet: Learning structured policy with graph neural networks," in *International conference on learning representations*, 2018.

[19] Y. Luo, M. Yao, and X. Xiao, "Gcnt: Graph-based transformer policies for morphology-agnostic reinforcement learning," *arXiv preprint arXiv:2505.15211*, 2025.

[20] M. Liu, D. Pathak, and A. Agarwal, "Locoformer: Generalist locomotion via long-context adaptation," *arXiv preprint arXiv:2509.23745*, 2025.

[21] X. Chen, Y. Gao, H. Liu, F. Yang, A. Ghadirzadeh, J. Yang, B. Liang, C. Zhang, T. L. Lam, and S.-C. Zhu, "Cross-robot behavior adaptation through intention alignment," *Science Robotics*, vol. 11, no. 112, p. eadv2250, 2026.

[22] S. Gupta, D. H. Salunkhe, and A. Billard, "Demonstrate once, execute on many: Kinematic intelligence for cross-robot skill transfer," *Science Robotics*, vol. 11, no. 113, p. eaea1995, 2026.

[23] J. Hwangbo, J. Lee, and M. Hutter, "Per-contact iteration method for solving contact dynamics," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 895–902, 2018.